

empty one that extends from `Backbone.Model`. This doesn't mean that it does nothing though! It simply means we don't need to add anything to the features Backbone already offers. We'll start with an empty model, and then store some data in an instance of it.

```
var Contact = Backbone.Model.extend({});
var graceHopper = new Contact({
  name: 'Grace Hopper'
});
console.log(graceHopper.get('name'));
```

The above code creates a Model for contacts called 'Contact', creates a new `Contact` variable with an attribute called `name`. We also demonstrate getting this name attribute out of the model and logging it to console.

Now let's demonstrate some of the juicier features of Backbone. We'll add code that listens to changes in the model, and outputs the raw JSON content of the model.

```
graceHopper.on('change', function() {
  console.log(graceHopper.toJSON());
});
graceHopper.set({'about': 'Inventor
of first compiler.'});
```

Let's go backwards with this. Notice how easy it is to set a new attribute on our object. You could as easily supply a new name attribute to change it; you could even supply multiple attributes at once.

The first bit of code that calls the `on` method is adding an event listener, that will call the supplied method when any attribute is added or changed. The method supplied here logs the JSON representation of our data to console using the `toJSON` method. You can have multiple statements and can even listen to changes in specific attributes if you want. For instance, if you just wanted to know when the `name` attribute has changed, you can replace the change above with `change:name`.

Models get even more interesting when server-side data comes into the picture. This is expertly handled by Backbone using just a single attribute: `urlRoot`. If you have server-side data accessible via a RESTful interface that provides JSON data, you're all set to go with Backbone. Let's say your server at 'http://myserver/' provides a list of contacts at 'http://myserver/contacts' and lets you access an individual contact by id at 'http://myserver/contacts/' you should set `urlRoot` to 'http://myserver/contacts'.

Once you have set the appropriate `urlRoot` Backbone can automatically fetch data and put it in the model for you to use. Let's look at an example:

```
var Contact = Backbone.Model.extend({
  urlRoot: 'http://myserver/contacts' });
```

*pointers

>> Interesting developer videos



*Android Studio Tutorials

>> The stable version of Android Studio is out, making it easier for app development in Android. Go and check out this playlist of Android Studio tutorials. Duration: 3:59:00

<http://dgit.in/AndroidStud>



*Linux Game Development

>> Keen on getting starting with development for games running on Linux? But before you do, watch this talk directed at Windows Steam developers on the same. Duration: 48:43

<http://dgit.in/LinuxGame>



*Star using material design

>> Watch the panel of developers answering various questions on the implementation of material design at the Chrome Dev Summit. Duration: 56:40

<http://dgit.in/MatDesPan>



*Develop in Swift

>> If you have been holding back on using Swift, Apple's programming language for Macs and iOS devices, get started with watching this video first. Duration: 1:05:12

<http://dgit.in/SwiftiOS>

```
var contact1 = new Contact({id:11});
contact1.fetch();
```

This code will fetch the data about the contact with id 11 from the server. You can modify this data and then call `contact1.save()` to post the changed data back to the server.

Backbone.Collection

Where Model deals with a single data object, a Collection deals with a bulk of data objects.

In our case, we can create an `AddressBook` collection that has a list of contacts. Once we create this Collection we can specify a `url` attribute that points to the list of contacts on the server, and it will automatically populate the collection with models when you call the `fetch` method on a Collection instance. You specify which Model this Collection is "collecting" via the `model` attribute.

Once you have a Collection of Model objects, you can use the `where` method to search for models matching certain attributes, fetch individual models based on their index in the Collection, or their id on the server. We demonstrate all of this in the following code sample.

```
var AddressBook = Backbone.Collection({
  url: 'http://myserver/contacts',
  model: Contact
});
var contacts = new AddressBook();
contacts.fetch();
var developers = contacts
  .where({'job': 'Developer'});
var person_at_position_11 = contacts.at(11);
var person_with_id_11 = contacts.get(11);
```

Backbone.Router

The final piece of the puzzle that we will look at is the Backbone Router. This is the module that makes navigating to different parts of your app work. It can map changes in the state of the app to changes in the URL, so it is possible to bookmark specific locations within the app. For instance, in a contact management application a user might want to bookmark a specific contact and go to it directly rather than needing to navigate to it every time.

Here is a sample router for such a contact management application:

```
var AppRouter = Backbone.Router.extend({
  routes: [
    'contacts', 'contactsList',
    'contacts/:id', 'contactDetail'
  ],
  contactsList: function() {...},
  contactDetail: function(id) {...}
});
var router = new AppRouter;
```